

Aim

To write a PL/SQL block using different control (if else, for loop, while loop...) for factorial of two numbers.

Procedure

Step 1 : Start

Step 2 : Initialize the necessary parameters

Step 3 : Develop the set of statements with the essential operational parameters.

Step 4 : Specify the individual operations to be carried out.

Step 5 : Execute the statements.

Step 6 : Stop

Output

Enter value of $n := 5$

old $4 : n \text{ number} := 2n;$

new $4 : n \text{ number} := 5;$

The factorial of a given number is : 120

Procedure successfully completed.

REG NO: 21TD0092
Y3/S5/B

NAME: VUDDAGIRI ABHINAYA
YEAR/SEM/SEC:

FACTORIAL OF A NUMBER

PROGRAM:

SQL> set serveroutput on;

declare

f number:=1;

i number;

n number:=&n;

begin

for i in 1..n

loop

f:=f*i;

end loop;

dbms_output.put_line('the factorial of given number is:'||f);

end;

/

Sd/-
29/10/23

Result:

Thus the PL/SQL block for factorial of two numbers are verified and executed.



EX.NO: 14

PL/SQL
SUM OF SERIES

19/10/23

Aim

To write a PL/SQL block using different control (if else, for loop, while loop, ...) for sum of series statements.

Procedure

- Step 1: Start
- Step 2: Initialize the necessary parameters
- Step 3: Develop the set of statements with the essential operational parameters.
- Step 4: Specify the individual operations to be carried out.
- Step 5: Execute the statements.
- Step 6: Stop.

output

SQL > set server output on;

SQL > |

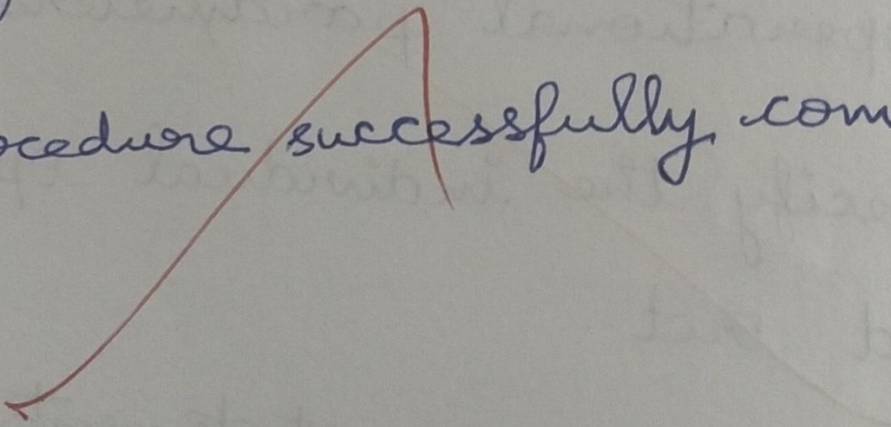
Enter value for n: 5

old 2: n number := &n;

new 2: n number is := 5;

The sum of series is: 15

PL/SQL procedure successfully completed.



REG NO: 21TD0092
Y3/S5/B

NAME: VUDDAGIRI ABHINAYA
YEAR/SEM/SEC:

SUM OF SERIES

PROGRAM:

```
SQL> set serveroutput on;
```

```
declare
```

```
n number:=&n;
```

```
i number;
```

```
begin
```

```
i:=n*(n+1);
```

```
n:=i/2;
```

```
dbms_output.put_line('the sum of series of given number is:'||n);
```

```
end;
```

```
/
```


By
20/10/23

Result:

Thus the PL/SQL block for sum of series
was verified and executed.



EX. NO: 15	PL/SQL FIBONACCI SERIES	26/10/23
------------	----------------------------	----------

Aim

To write a PL/SQL block using different control (if else, for loop, while loop, ...) for Fibonacci series statements:

Procedure

- Step 1: Start
- Step 2: Initialize the necessary parameters
- Step 3: Develop the set of statements with the essential operational parameters.
- Step 4: Specify the individual operations to be carried out.
- Step 5: Execute the statements.
- Step 6: Stop

Output:

Enter value for n: 5

old 4: n number is: = 2 n;

new 4: n number is: = 5;

0

1

2

3

PL/SQL procedure successfully completed.

REG NO: 21TD0092
Y3/S5/B

NAME: VUDDAGIRI ABHINAYA YEAR/SEM/SEC:

FIBONACCI SERIES

PROGRAM:

```
SQL>set serveroutput on;
```

```
declare
```

```
i number;
```

```
c number;
```

```
n number:=&n;
```

```
a number:=-1;
```

```
b number:=1;
```

```
begin
```

```
for i in 1..n
```

```
loop
```

```
c:=a+b;
```

```
dbms_output.put_line(c);
```

```
a:=b;
```

```
b:=c;
```

```
end loop;
```

```
end;
```

```
/
```


26/06/23
Result:

Thus the PL/SQL for Fibonacci series was
verified and executed.



Aim
To write a PL/SQL block using different control (if else, for loop, while loop, ...) for palindrome statements.

Procedure

- Step 1: Start
- Step 2: Initialize the necessary parameters
- Step 3: Develop the set of statements with the essential operational parameters.
- Step 4: Specify the individual operations to be carried out.
- Step 5: Execute the statements
- Step 6: Stop

Output:

Enter value for str1: madam

old4: str1 varchar(10): 'madam';

new4: str1 varchar(10): 'madam';

madam is a palindrome

PL/SQL procedure successfully completed.

REG NO: 21TD0092
Y3/S5/B

NAME: VUDDAGIRI ABHINAYA YEAR/SEM/SEC:

PALINDROME SERIES

PROGRAM:

```
SQL>set serveroutput on;
```

```
declare
```

```
len number;
```

```
a integer;
```

```
str1 varchar(10):='&str1';
```

```
str2 varchar(10);
```

```
begin
```

```
len:=length(str1);
```

```
a:=len;
```

```
for i in 1..a
```

```
loop
```

```
str2:=str2||substr(str1,len,1);
```

```
len:=len-1;
```

```
end loop;
```

```
if(str1=str2)then
```

```
dbms_output.put_line(str1||' is a palindrome');
```

```
else
```

```
dbms_output.put_line(str1||' is not a palindrome');
```

```
end if;
```

```
end;
```

```
/
```


24/2/23
Result:

Thus the PL/SQL for palindrome was verified and executed.

Aim

To write a PL/SQL program using trigger to insert, update and delete on table.

Algorithm

- Step 1: Start the program.
- Step 2: Declare all necessary variables.
- Step 3: Create a package as 'packs'.
- Step 4: End the page.
- Step 5: After creating package, create a display in the procedure.
- Step 6: Check the number is prime or not in the if else statement.
- Step 7: Check whether the given number is odd or even in the if else statement.
- Step 8: Add check whether the given number is positive or negative using the if else statement.

REG NO: 21TD0092 NAME: VUDDAGIRI ABHINAYA
YEAR/SEM/SEC: Y3/S5/B

PACKAGES

SOURCE CODE:

PACKAGE EXCEPTION:

```
create or replace package packs is  
procedure display;  
function prime(n number) return varchar;  
function odd(n number) return varchar;  
function positive(n number) return varchar;  
end packs;
```

PACKAGE BODY:

```
create or replace package body packs as  
procedure display is  
avarchar(15);  
m number;  
begin  
m:=&m;  
a:=prime(m);  
if a=prime(m)then  
dbms_output.put_line(m||'is a prime');  
else  
dbms_output.put_line(m||'is not prime');  
end if;
```



```
a:=odd(m);
if a='odd' then
dbms_output.put_line(m||'is odd');
else
dbms_output.put_line(m||'is even');
end if;
a:=positive(m);
if a='positive' then
dbms_output.put_line(m||'is a positive number');
else
dbms_output.put_line(m||'is a negative number');
end if;
end display;
function prime(n number) return varchar is
i number;
flag number;
begin
flag:=0;
for i in 2..n/2
loop
if mod(n,i)=0 then
flag:=1
return 'not prime';
end if;
end loop;
if flag=0 then
return 'prime';
end if;
```


--	--	--

end prime;

function odd(n number) return varchar is

begin

if $n > 0$ then

return 'positive';

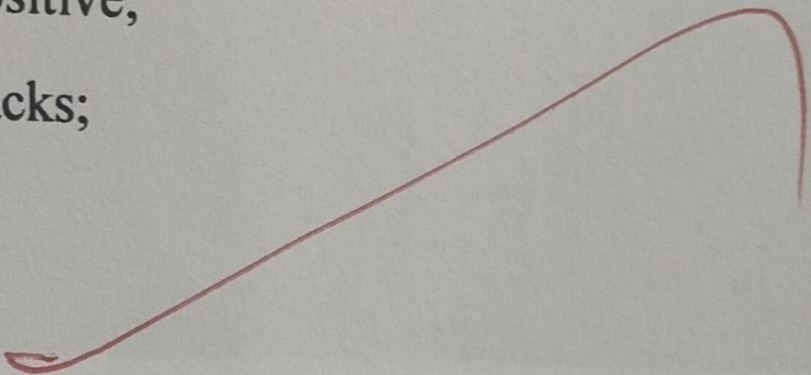
else

return 'negative';

end if;

end positive;

end packs;



OUTPUT:

```
31 /  
Enter value for n: 7  
old 6: n number:=&n;  
new 6: n number:=7;  
7 is a prime  
7 is odd  
7 is a positive number  
  
PL/SQL procedure successfully completed.  
  
SQL> █
```

dy
2/1/23
Result:

Thus the PL/SQL triggers for package
has been executed and verified successfully.

Aim

To write a PL/SQL block that deletes the record and display the number of records deleted.

Procedure

- Step 1: Start the program
- Step 2: Declare the variables
- Step 3: Open a loop to fetch
- Step 4: Exit the loop when C4 is not found
- Step 5: Assign the $n = n + 1$;
- Step 6: End loop
- Step 7: Display the detailed and remaining of records.
- Step 8: End the program.

REG NO: 21TD0092 NAME: VUDDAGIRI ABHINAYA YEAR/SEM/SEC:
Y3/S5/B

CURSOR

SOURCE CODE:

DELETING THE RECORD USING CURSOR

SQL> declare

cursor c is select * from vehicle1;

a c % rowtype;

n number:=0;

begin

open c;

loop

fetch c into a;

exit when c% notfound;

if a.vehicle_no=521 then

delete from vehicle1 where vehicle_no=521;

n:=n+1;

end if;

end loop;

dbms_output.put_line('deleted record'||n);

dbms_output.put_line('remaining records'||(c % rowcount-n));

close c;

commit;

end;

/

OUTPUT:

VEHICLE_NO	VEHIC	SOURCE	DESTINATIO	SEAT_COST	ARR_TIME
521	duke	pondy	cuddalore	52	12
623	prtc	pondy	banglore	35	2
632	mnc	bangalore	chennai	23	1

SQL>

PL/SQL procedure successfully completed.

SQL> select * from vechicle1;

VEHICLE_NO	VEHICLE	SOURCE	DESTINATIO	SEAT_COST	ARR_TIME
623	prtc	pondy	banglore	35	2
632	mnc	bangalore	chennai	23	1

SQL>

Result:

Thus the cursor procedure for calculating the payroll process has been executed successfully.

